

HLD Chapter 2: Back-of-the-Envelope Estimations Masterclass

Core Learning Objective: Master the mathematical formulas, rules of thumb, and conversion shortcuts required to estimate QPS, storage, bandwidth, and cache sizes in under 5 minutes during a systems design interview.

1. Golden Rules of Thumb & Approximation Constants

In an interview, **never spend time doing manual long division with decimals**. Interviewers expect clean, fast order-of-magnitude estimations (10^x).

The 86,400 Approximation Rule

$1 \text{ day} = 24 \text{ hours} \times 60 \text{ minutes} \times 60 \text{ seconds} = 86,400 \text{ seconds} \approx 10^5 \text{ seconds}$

$\text{QPS} = \frac{\text{Daily Requests}}{10^5}$

- **10 Million requests/day** $\approx \frac{10^7}{10^5} = 100 \text{ QPS}$
- **100 Million requests/day** $\approx \frac{10^8}{10^5} = 1,000 \text{ QPS}$
- **1 Billion requests/day** $\approx \frac{10^9}{10^5} = 10,000 \text{ QPS}$

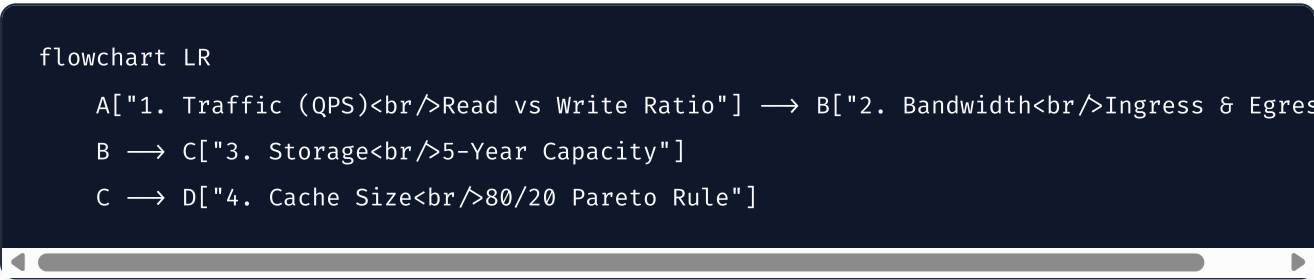
Peak Traffic Multiplier

Systems must be provisioned for **Peak QPS**, not average QPS: $\text{Peak QPS} = \text{Average QPS} \times 2 \text{ (or } 3 \text{ for spiky apps like Twitter/Uber)}$

2. Storage & Memory Units Mapping

Power of 2	Power of 10	Value	Unit	Concrete Example
2^{10}	10^3	1,024	Kilobyte (KB)	Small JSON payload / metadata
2^{20}	10^6	1,048,576	Megabyte (MB)	High-res compressed photo
2^{30}	10^9	1,073,741,824	Gigabyte (GB)	Modern RAM memory stick
2^{40}	10^{12}	10^{12}	Terabyte (TB)	Hard drive disk capacity
2^{50}	10^{15}	10^{15}	Petabyte (PB)	Datacenter storage tier

3. The 4-Step Estimation Blueprint



Step 1: Traffic Estimation (Read vs Write QPS)

1. Identify **Daily Active Users (DAU)**.
2. Determine user actions per day (e.g. 10 reads, 1 write per user).
3. Compute Read QPS and Write QPS.

Step 2: Storage Requirements (5-Year Capacity)

$$\text{Daily Storage} = \text{Writes/Day} \times \text{Size per Record}$$
$$\text{5-Year Storage} = \text{Daily Storage} \times 365 \times 5 \approx \text{Daily Storage} \times \mathbf{2,000}$$

Step 3: Network Bandwidth (Ingress & Egress)

- **Ingress (Incoming):** $\text{Write QPS} \times \text{Average Write Payload Size}$
- **Egress (Outgoing):** $\text{Read QPS} \times \text{Average Read Payload Size}$

Step 4: Cache Memory Sizing (The 80/20 Pareto Principle)

In almost every consumer system, **\$20%\$ of daily read requests account for \$80%\$ of total read volume** (the "hot" data).
$$\text{Cache RAM} = \text{Daily Total Read Data Volume} \times \mathbf{0.20}$$

4. End-to-End Walkthrough: Twitter-Scale Social Platform

Assumptions & Given Metrics:

- **DAU:** \$300 \text{ Million}\$ active users.
- **Writes:** Each user posts an average of \$2\$ tweets per day.
- **Reads:** Each user views an average of \$20\$ tweets per day.
- **Media:** \$10\%\$ of tweets contain an image (\$200 \text{ KB}\$), \$90\%\$ are pure text (\$200 \text{ bytes}\$).

1. QPS Calculation:

- **Total Writes/Day:** $300 \text{M} \times 2 = 600 \text{ Million tweets/day}$. $\text{Write QPS} = \frac{600 \times 10^6}{10^5} = \mathbf{6,000 \text{ QPS}}$ $\text{Peak} = 6,000 \times 2 = \mathbf{12,000 \text{ QPS}}$
- **Total Reads/Day:** $300 \text{M} \times 20 = 6 \text{ Billion reads/day}$. $\text{Read QPS} = \frac{6 \times 10^9}{10^5} = \mathbf{60,000 \text{ QPS}}$ $\text{Peak} = 60,000 \times 2 = \mathbf{120,000 \text{ QPS}}$
- **Read-to-Write Ratio:** \$10 : 1\$ (Read-heavy architecture).

2. Storage Calculation:

- **Average Tweet Size:** $(0.90 \times 200 \text{ B}) + (0.10 \times 200 \text{ KB}) \approx 180 \text{ B} + 20 \text{ KB} \approx \mathbf{20 \text{ KB}}$.
- **Daily Storage:** $600 \text{M writes} \times 20 \text{ KB} = 12 \text{ TB / day}$.
- **5-Year Storage:** $12 \text{ TB} \times 365 \times 5 \approx \mathbf{22 \text{ PB}}$ (Petabytes).

3. Cache Sizing (80/20 Rule):

- **Daily Read Volume:** $6 \text{ Billion reads} \times 20 \text{ KB} = 120 \text{ TB / day}$.
- **Cache 20% of Hot Daily Data:** $120 \text{ TB} \times 0.20 = \mathbf{24 \text{ TB of RAM}}$.
- If each Redis cache server has \$256 \text{ GB}\$ of RAM: $\text{Number of Redis Nodes} = \frac{24 \text{ TB}}{0.256 \text{ TB}} \approx \mathbf{94 \text{ Cache Instances}}$.